Instruction Set Architecture (ISA)

"User's manual for the computer"

Instructor: Jeongseob Ahn (안정섭)

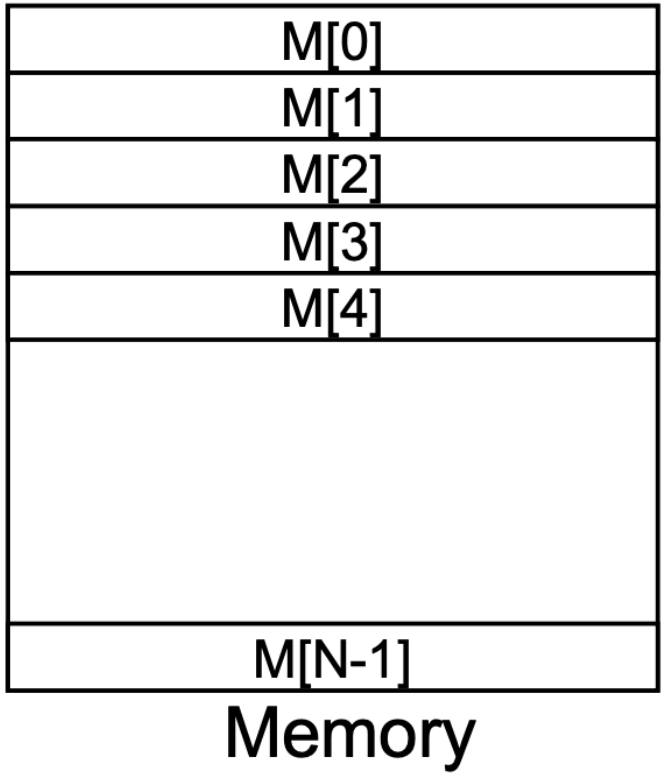
E-mail: jsahn@csl.korea.ac.kr
Hompage: https://jeongseob.github.io

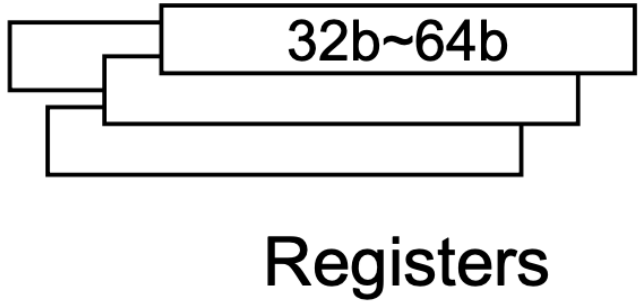
What are specified/decided in an ISA?

- Data format and size
  - character, binary, decimal, floating point, negatives
- "Programmer Visible State" **(a.k.a. architectural state)**
  - memory, registers, program counter (PC), etc.
- Instructions: how to **change** the programmer visible state?
  - What to perform and what to perform next
  - Where the operands are
- Instruction-to-binary (or vice versa) encoding
- How to interface with the outside world?
- Protection and privileged operations
- Software conventions

Very often you compromise immediate optimality for future scalability and compatibility

Programmer Visible State





Program Counter (32b~64b)

General Instruction Classes

- Arithmetic and logical operations (e.g., add, sub, and, or)
  1) Fetch operands from specified locations
  2) Compute a result as a function of the operands
  3) Store result to a specified location
  4) Update PC to the next sequential instruction
- Data movement operations (e.g., move, load, store)
  1) Fetch operands from specified locations
  2) Store operand values to specified locations
  3) Update PC to the next sequential instruction
- Control flow operations (e.g., branch, jump)
  1) Fetch operands from specified locations
  2) Compute a branch condition and a target address
  3) If "branch condition is true" then PC ← target address
     else PC ← next seq. instruction

Mainstream ISAs

x86

Designer	Intel, AMD
Bits	16-bit, 32-bit and 64-bit
Introduced	1978 (16-bit), 1985 (32-bit), 2003 (64-bit)
Design	CISC
Type	Register-memory
Encoding	Variable (1 to 15 bytes)
Endianness	Little

Macbooks & PCs
(Core i3, i5, i7, M)

ARM architectures

Designer	ARM Holdings
Bits	32-bit, 64-bit
Introduced	1985; 31 years ago
Design	RISC
Type	Register-Register
Encoding	AArch64/A64 and AArch32/A32 use 32-bit instructions, T32 (Thumb-2) uses mixed 16- and 32-bit instructions. ARMv7 user-space compatibility[1]
Endianness	Bi (little as default)

Smartphone-like devices
(iPhone, iPad, Raspberry Pi)

Operations of Computer Hardware

Instruction Set

- The repertoire of instructions of a computer
- Different computers have different instruction sets
  - But with many aspects in common
- Early computers had very simple instruction sets
  - Simplified implementation
- Many modern computers also have simple instruction sets

The RISC-V Instruction Set

- Used as the example throughout the book
- Developed at UC Berkeley as open ISA
- Now managed by the RISC-V Foundation (riscv.org)
- Typical of many modern ISAs
  - See RISC-V Reference Data tear-out card
- Similar ISAs have a large share of embedded core market
  - Applications in consumer electronics, network/storage equipment, cameras, printers, …

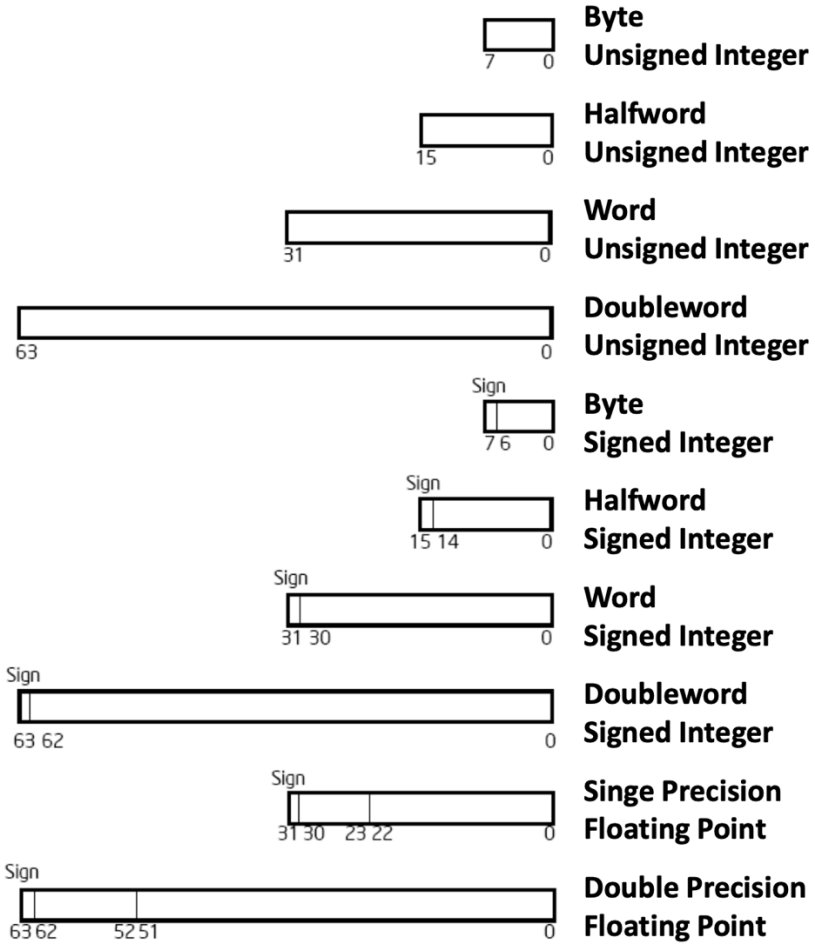
RISC-V ISAs

- Three base integer ISAs, one per address width
  - RV32I, RV64I, RV128I
  - RV32I: only 40 instruction defined
  - RV32E: Reduced version of RV32I with 16 registers for embedded systems
- Standard extentions
  - Standard RISC encoding in a fixed 32-bit instruction format

Extension	Description
M	Integer multiplication and division
A	Atomic instructions
F	Single-precision FP
D	Double-precision FP
G	General-purpose (=IMAFD)
Q	Quad-precision FP
H	Hypervisor extension
V	Vector operations

Data Types

- Integer data of 1, 2, 4, or 8 bytes
  - Data values
  - Addresses (untyped pointers)
- Floating point data of 4 or 8 bytes (with F or D extension)
- No aggregated types such as arrays or structures
  - Just contiguously allocated bytes in memory



RISC-V Registers

Register	Name	Use	Saver
x0	zero	The constant value 0	N.A.
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	--
x4	tp	Thread pointer	--
x5 – x7	t0 – t2	Temporaries	Caller
x8	s0 / fp	Saved register / Frame pointer	Callee
x9	s1	Saved registers	Callee
x10 – x11	a0 – a1	Function arguments/Return values	Caller
x12 – x17	a2 – a7	Function arguments	Caller
x18 – x27	s2 – s11	Saved registers	Callee
x28 – x31	t3 – t6	Temporaries	Caller

Arithmetic Operations

- Add and subtract, three operands
  - Two sources and one destination

```
add   a, b, c    // a = b + c
sub   a, b, c    // a = b - c
```

- All arithmetic operations have this form
- **Design Principle 1: Simplicity favors regularity**
  - Regularity makes implementation simpler
  - Simplicity enables higher performance at lower cost

Arithmetic Example

- C code:

```
f = (g + h) - (i + j);
```

- Compiled RISC-V code (assembly):

Operands of Computer Hardware

Register Operands

- Arithmetic instructions use register operands
- RISC-V has a 32 x 64-bit register file
  - Use for frequently accessed data
  - 64-bit data is called a ”doubleword”
    - 32 general purpose registers `x0` to `x31`
      - `RV32I` → 32-bits, `RV64I` → 64-bits
  - 32-bit data is called a “word”
- **Design Principle 2: Smaller is faster**
  - (c.f.) Main memory: millions of locations

Register Operand Example

- C code:

```
f = (g + h) - (i + j);
```

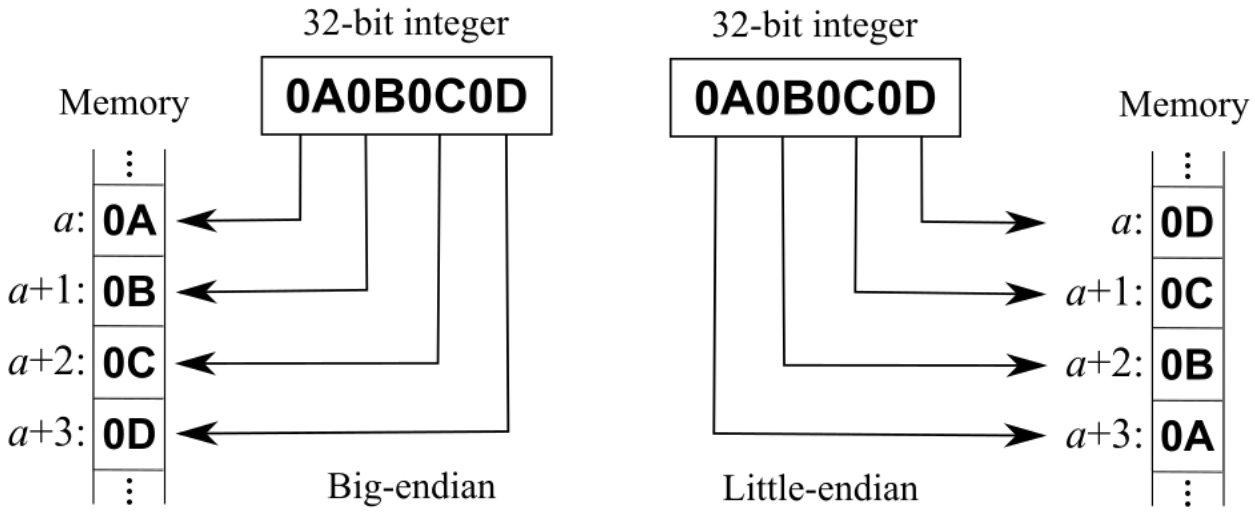
  - Assume `f`, `g`, `h`, `i`, and `j` are stored in `x19`, `x20`, `x21`, `x22`, and `x23`, respectively
  - Assume we have two free registers: x5 and x6

- Compiled RISC-V code:

Memory Operands

- Main memory used for composite data
  - Arrays, structures, dynamic data
- To apply arithmetic operations
  - Load values from `memory` into `registers`
  - Store result from `register` to `memory`
- Memory is byte addressed
  - Each address identifies an 8-bit (byte)
- RISC-V is Little Endian
  - Least-significant byte at least address of a word
  - (c.f.) Big Endian: most-significant byte at least address

Big Endian vs. Little Endian



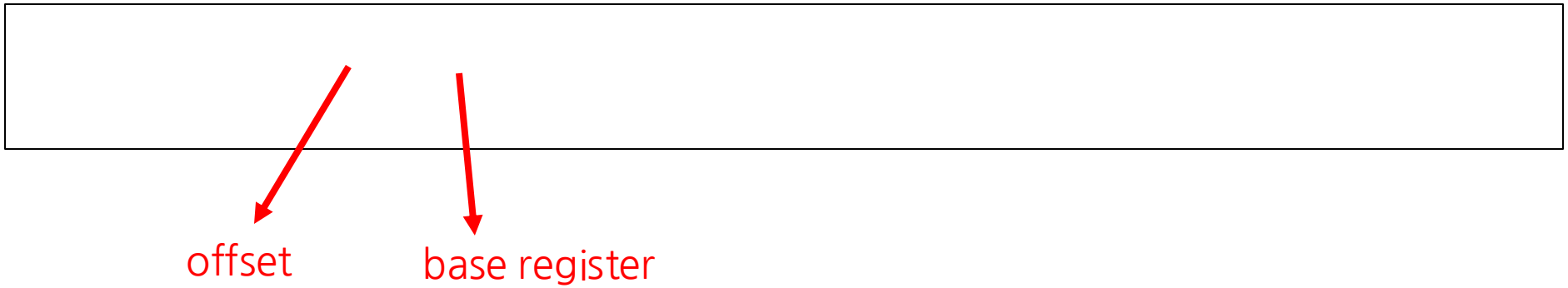
Memory Operand Example (1)

- C code:

```
g = h + A[8];
```

  - `g` in `$x20`, `h` in `x21`, base address of `A` in `x22`

- Compiled RISC-V code:
  - Index 8 requires offset of 64
    - 8 bytes per doubleword



Memory Operand Example (2)

- C code:

```
A[12] = h + A[8];
```

  - `h` in `x21`, base address of `A` in `x22`

- Compiled RISC-V code:
  - Index 8 requires offset of 64
    - 8 bytes per doubleword

```
ld  x9, 64(x22) // load doubleword
add x9, x21, x9
```

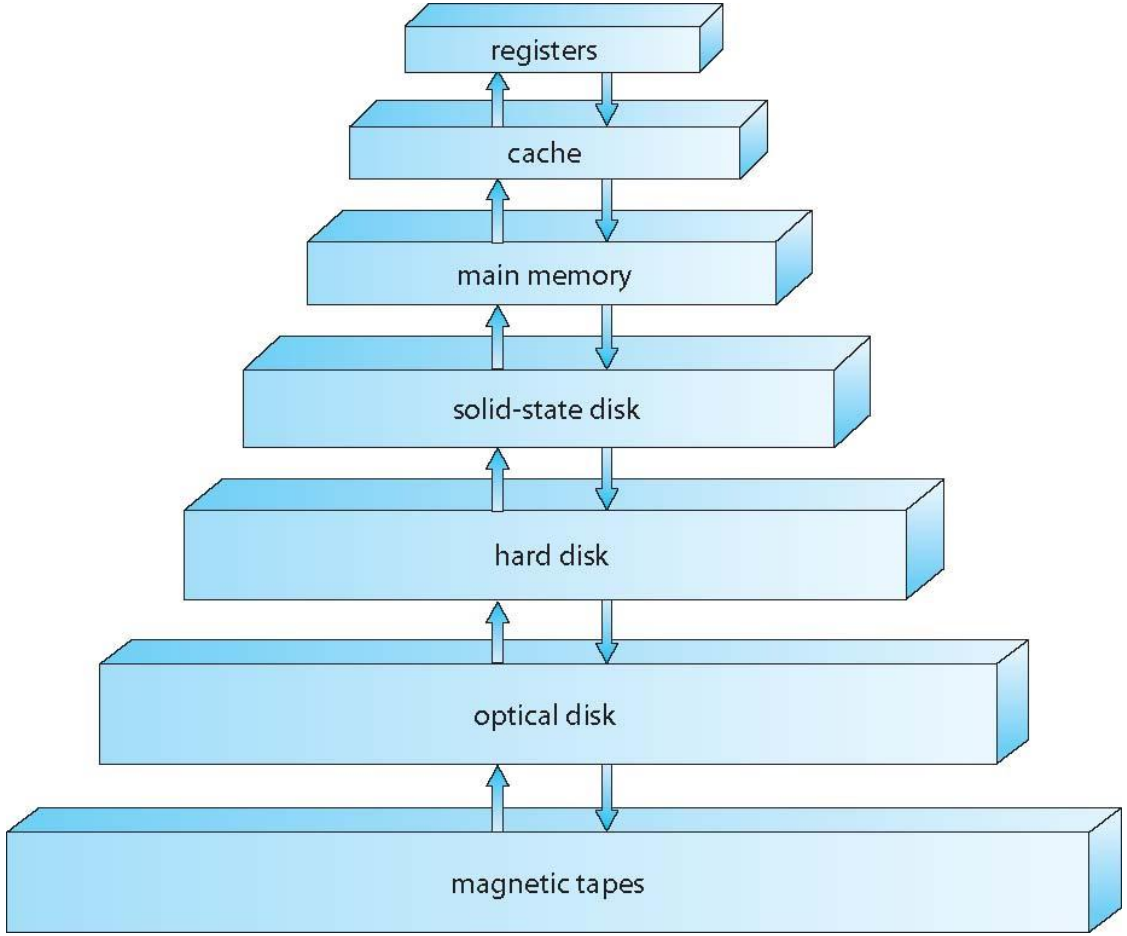
Byte/Halfword/Word Operations

- RISC-V byte/halfword/word load/store
  - Load byte/halfword/word: Sign extend to 64 bits in `rd`
    - `lb rd, offset(rs1)`
    - `lh rd, offset(rs1)`
    - `lw rd, offset(rs1)`
  - Load byte/halfword/word unsigned: Zero extend to 64 bits in `rd`
    - `lbu rd, offset(rs1)`
    - `lhu rd, offset(rs1)`
    - `lwu rd, offset(rs1)`
  - Store byte/halfword/word: Store rightmost 8/16/32 bits
    - `sb rs2, offset(rs1)`
    - `sh rs2, offset(rs1)`
    - `sw rs2, offset(rs1)`

Register vs. Memory

- Registers are faster to access than memory
- Operating on memory data requires loads and stores
  - More instructions to be executed
- Compiler must use registers for variables as much as possible
  - Only spill to memory for less frequently used variables
  - Register optimization is important

Memory Hierarchy



Immediate Operands

- Constant data specified in an instruction

```
addi  x22, x22, 4
```

- No subtract immediate instruction
  - Just use a negative constant

```
addi  x22, x22, -1
```

- **Design Principle 3: Make the common case fast**
  - Small constants are common
  - Immediate operand avoids a load instruction

Constant

- The constant zero
  - Register `x0` is the constant 0
    - Cannot be overwritten
  - Useful for common operations
    - E.g., move between registers

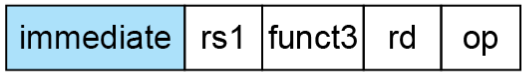
```
add   x7, x22, x0
```

Summary of RISC-V Design Principles

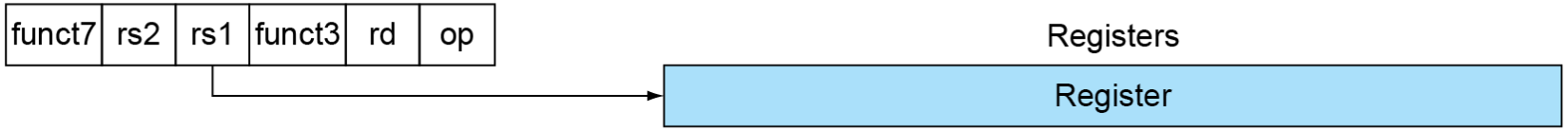
1. Simplicity favors regularity
   - Fixed size instructions
   - Small number of instruction formats
   - OPcode is always the lower 7 bits
2. Smaller is faster
   - Limited instruction set
   - Limited number of registers in register file
   - Limited number of addressing modes
3. Make the common case fast
   - Arithmetic operands from the register file
   - Allow instructions to contain immediate operands

RISC-V Addressing

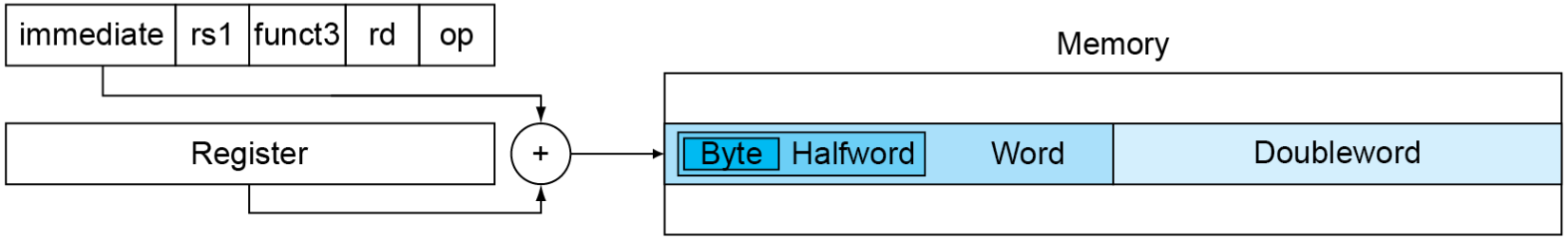
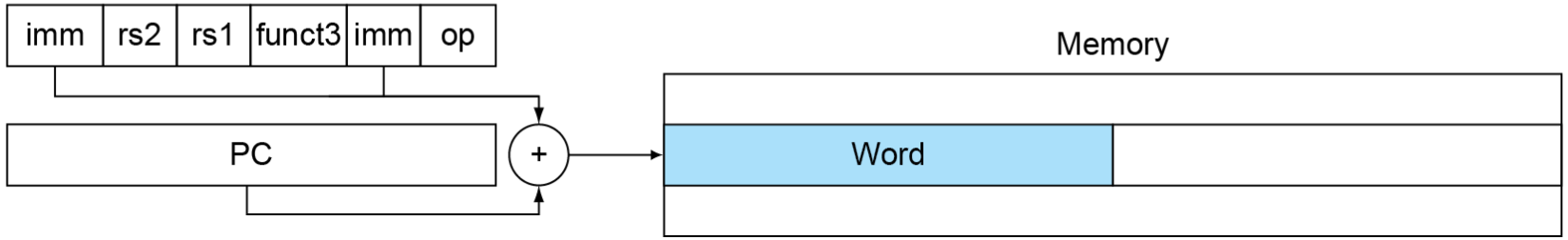
1. Immediate addressing



2. Register addressing



3. Base addressing



4. PC-relative addressing